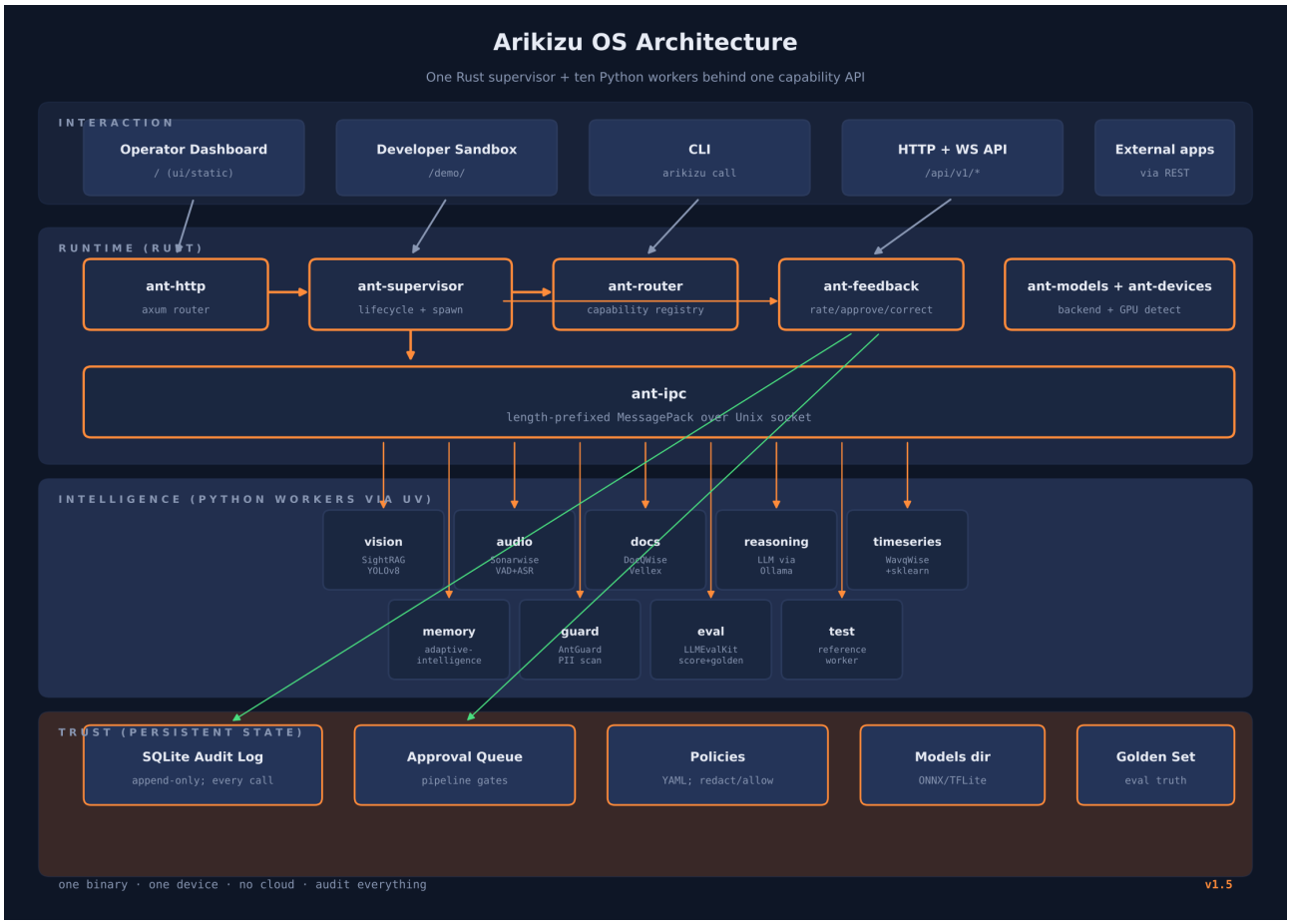


# Arikizu OS

## ARCHITECTURE REFERENCE

v1.5 · October 2026 · Venkatkumar Rajan

# Arikizu Architecture




One Rust binary that supervises ten Python workers. One HTTP surface for every capability. One SQLite file for every event. One device, one install, no cloud.

What follows: how it fits together, where state lives, where the moving parts are. For why it is called an OS, see `WHY_OS.md`. For how it sits on top of Raspberry Pi OS, see `HOW_IT_RUNS.md`.

---

## 1. Four zones

The system splits cleanly into four zones. Each zone has one job and a single, deliberate boundary with the next.

| Zone | What it does | Tech |
|---|---|---|
| **Interaction** | How humans and apps talk to Arikizu | HTTP, WebSocket, CLI, browser |
| **Runtime** | Supervision, routing, policy, device detection | Rust (9 crates) |
| **Intelligence** | The actual AI work | Python (10 workers, uv) |
| **Trust** | Everything we promise to remember or enforce | SQLite, YAML, filesystem |

Picture at the top of this file shows the same split visually.

---

## 2. Interaction zone (what the user sees)

Four entry points. All of them land on the same `/api/v1/*` surface.

### 2.1 Operator dashboard ( `/` )

Served from `ui/static/index.html` by `ant-http` 's fallback service. Zero build step, no npm. Pulls live data from three endpoints:

| Endpoint | What it shows on the dashboard |
|---|---|
| `GET /api/v1/status` | Device chip, worker count, uptime |
| `GET /api/v1/audit?limit=5` | Recent activity feed |
| `GET /api/v1/capabilities` | How many capabilities exist |

Mobile responsive. Sidebar collapses under 700 px.

### 2.2 Developer sandbox ( `/demo/` )

Served from `demo/web/` . One HTML file per capability. Each one is a few hundred lines of plain JS that `fetch` 's the matching `/api/v1/call/<cap>` endpoint. This is the surface used for raw testing and demos.

### 2.3 CLI ( `arikizu call ...` )

The `ant-cli` crate speaks the same HTTP API. Example:

```
arikizu call test.echo '{"message":"hello"}'
```

That is `POST /api/v1/call/test/echo` with the payload wrapped in a `{"payload":...}` envelope.

### 2.4 External apps

Anything that can `POST` JSON can drive Arikizu. The capability API is stable; a shell script, a Rust service, or a Jupyter notebook all look the same from the server side.

---

## 3. Runtime zone (the Rust core)

Nine crates in one Cargo workspace. Each one is small and does one thing.

```
crates/
  ant-common/      core types, errors, result type
  ant-ipc/         length-prefixed MessagePack over Unix socket
  ant-router/      capabilities.toml loader, lookup by name
  ant-supervisor/  worker lifecycle: spawn, call, shutdown, restart
  ant-models/      model manifest, SHA-256 verification, backend pick
  ant-devices/     CPU / GPU / Jetson detection
  ant-feedback/    rate, correct, approve, reject, annotate
  ant-http/        axum REST + WebSocket + static file serving
  ant-cli/         the `arikizu` binary
```

### 3.1 ant-common (foundation)

Types every other crate uses. Errors ( `ArikizuError` ), result ( `Result<T>` ), the pipeline config shape ( `ArikizuConfig` ), feedback labels, event enums. Nothing async here. Nothing I/O. Pure data.

### 3.2 ant-ipc (the pipe)

The Rust side of the IPC. Length-prefixed MessagePack frames over a Unix domain socket. The Python SDK ( `arikizu-worker-sdk` ) speaks the same wire format on the other end. Chosen over gRPC because:

- No code generation step.
- Smaller binary.
- MessagePack is one Python import, one Rust crate.
- Length-prefix is easy to debug.

### 3.3 ant-router (capability registry)

Loads `capabilities.toml` at boot. Every capability declares:

```
[vision_detect]
name       = "vision.detect"
worker     = "vision"
timeout_ms = 5000
priority   = 10
```

The router is a `HashMap<String, CapabilitySpec>` with a few helpers ( `by_worker` , `names` , `get` ). 29 capabilities registered today.

### 3.4 ant-supervisor (the heart)

Owns every worker handle. For each worker in `arikizu.yaml` :

1. `spawn()` : forks a Python subprocess ( `uv run --package <worker> python -m <worker>` ), binds a Unix socket, waits for the worker to connect (60 s timeout for first-run dep downloads).
2. `call(cap, payload, timeout)` : writes a MessagePack frame to the socket, awaits the response, returns JSON.

3. `shutdown()` : sends a close frame, waits for the subprocess, SIGKILLs if it does not go.
4. Broadcasts worker state transitions on a `tokio::sync::broadcast` channel for the WebSocket event bus.

Also runs the pipeline executor ( `run_pipeline` ): walks a list of stages, honors `feedback.approve` gates (blocks the stage until the approval queue sees an approve or reject), writes a run record to the audit log.

### 3.5 ant-models (model selection)

Reads `models.toml` manifest, verifies SHA-256 of each file on disk, picks a backend per device. The decision table lives in `backend_selection.rs` :

| Platform | Backend chain |
|---|---|
| Jetson (any Orin) | TensorRT -> onnxruntime-gpu -> CPU |
| Linux x86_64 + NVIDIA | onnxruntime-gpu (CUDA) -> CPU |
| Mac (Apple Silicon) | onnxruntime (CoreML EP) -> CPU |
| Pi 4 / 5 | onnxruntime (CPU) |
| Everything else | onnxruntime (CPU) |

Each worker's `.backends` capability lets the dashboard see what was picked for that worker.

### 3.6 ant-devices (hardware probe)

Reads `/proc/cpuinfo` , `/proc/meminfo` , `nvidia-smi` , and the Jetson device tree. Returns `DeviceInventory { platform, is_jetson, cpus, gpus, total_memory_mb }` . The dashboard chip ("Raspberry Pi", "Jetson", "Mac") is driven from this.

### 3.7 ant-feedback (human in the loop)

Four tables in SQLite:

| Table | Rows written when |
|---|---|
| `ratings` | `/feedback/rate` is called |
| `corrections` | `/feedback/correct` is called |
| `approvals` | A pipeline stage hits a `feedback.approve` gate |
| `annotations` | `/feedback/annotate` is called |

The pending-approvals queue is an in-memory `HashMap<Uuid, PendingApproval>` backed by `approvals` on restart. Dashboard reads it via `GET /api/v1/feedback/pending` .

### 3.8 ant-http (the surface)

An `axum` router with 14 routes, grouped:

```
GET  /                          operator dashboard (fallback_service)
GET  /demo/*                    developer sandbox  (nest_service)
GET  /health                    liveness probe
GET  /api/v1/status             runtime summary
GET  /api/v1/devices            hardware inventory
GET  /api/v1/capabilities       capability list
GET  /api/v1/workers            worker list
POST /api/v1/call/:domain/:op   the only API that matters
GET  /api/v1/audit              tail of audit log
GET  /api/v1/events             WebSocket event feed
POST /api/v1/feedback/*         5 feedback endpoints
GET  /api/v1/pipeline/config    pipeline YAML
POST /api/v1/pipeline/run       kick off a pipeline run
GET  /api/v1/runtime            aggregated per-worker runtime info
POST /api/v1/stop               graceful shutdown
```

All JSON. All synchronous except `/events`. All documented in `docs/MODULES.md` with request / response shapes.

### 3.9 ant-cli (the binary)

`clap`-driven CLI. Subcommands:

```
arikizu run             launch supervisor + HTTP server (what systemd runs)
arikizu call <cap>      POST /api/v1/call via reqwest
arikizu status          GET /api/v1/status, pretty-print
arikizu stop            POST /api/v1/stop
arikizu check           runtime health probe
arikizu models list     list models known from models.toml
```

Everything the CLI does is a thin HTTP client over `127.0.0.1:7000`. No shared state with the running process.

---

## 4. IPC layer (the pipe between Rust and Python)

The one place where Rust and Python meet.

### 4.1 Wire format

```
[4 bytes big-endian length N][N bytes MessagePack body]
```

Body is a MessagePack map. Request:

```
{
  "id":       <uuid v4>,
  "capability": "vision.detect",
  "payload":   { ... any JSON ... },
  "deadline_ms": 5000
}
```

Response:

```
{
  "id":    <same uuid>,
  "ok":    true|false,
  "result": { ... } | null,
  "error":  "..." | null
}
```

### 4.2 Transport

Unix domain socket at `/tmp/arikizu-<worker>-<uuid>.sock` (one per worker, per supervisor run). Why not TCP:

- No port management.
- Filesystem-permission security (owner-only).
- Faster (no TCP stack overhead) on the Pi.
- Harder to accidentally expose to the network.

### 4.3 Python side (arikizu-worker-sdk)

Every worker imports this one SDK. Boilerplate it hides:

- Socket setup, length-prefix framing, MessagePack encode / decode
- Capability registration ( `@capability("vision.detect")` )
- Timeout handling, error wrapping
- Backend auto-selection (ONNX / TFLite / OpenVINO / TensorRT / CoreML)
- Graceful shutdown on SIGTERM

Writing a new worker is: install the SDK, decorate functions, run. ~30 lines for a working worker, including imports.

---

## 5. Intelligence zone (Python workers)

Ten workers, each in its own uv package under `workers/`.

| Worker | Wraps | Capabilities |
|---|---|---|
| `test` | nothing (reference impl) | `test.echo`, `test.sleep`, `test.fail` |
| `vision` | SightRAG + a compact vision model | `vision.detect`, `vision.segment`, `vision.track`, `vision.backends` |
| `audio` | Sonarwise + a VAD model + the ASR model | `audio.vad`, `audio.transcribe`, `audio.backends` |
| `docs` | DocQWise + Vellex | `docs.extract`, `docs.search` |
| `reasoning` | a local LLM endpoint | `reasoning.answer`, `reasoning.summarize`, `reasoning.complete` |
| `timeseries` | WavqWise + classical ML libraries + scipy | `timeseries.forecast`, `timeseries.anomaly` |
| `memory` | adaptive-intelligence | `memory.update`, `memory.recall`, `memory.reward` |
| `guard` | AntGuard (regex v0.1) | `guard.scan`, `guard.redact` |
| `eval` | LLMEvalKit | `eval.score`, `eval.golden_run` |
| `feedback` | n/a (handled by ant-feedback) | (Rust-side only) |

Each worker:

- Runs under its own uv venv, so dependency conflicts can't happen.
- Is CPU-bound by default; GPU use is backend-driven.
- Can be swapped out of `arikizu.yaml` with one line of config.
- Loads models from `models/` only on first call (lazy init).

Full catalog: `MODELS.md`.

---

## 6. Trust zone (what survives a restart)

Persistent state has a dedicated zone because it is what Arikizu promises to its users. Four kinds.

### 6.1 SQLite audit log ( `data/audit.sqlite` )

One table:

```
CREATE TABLE events (
    id         INTEGER PRIMARY KEY AUTOINCREMENT,
    ts         INTEGER NOT NULL,            -- epoch ms
    kind       TEXT    NOT NULL,            -- e.g. "vision.detect"
    actor      TEXT,                        -- user, system, worker name
    ref        TEXT,                        -- correlation id
    body       TEXT    NOT NULL             -- JSON payload
);
CREATE INDEX idx_events_ts ON events(ts DESC);
CREATE INDEX idx_events_kind ON events(kind);
```

Append-only. Every HTTP call, every pipeline stage, every feedback event. The dashboard's "recent activity" reads from here. Operators can grep this file for anything that ever happened.

### 6.2 Approval queue (in-memory + SQLite)

Pipeline stages marked `require_approval: true` block until a human hits approve or reject. The pending set lives in memory for speed; decisions are written to the `approvals` SQLite table on commit so a restart doesn't lose the record of what was approved.

### 6.3 Policies ( `policies/*.yaml` )

Declarative rules the supervisor consults before dispatch:

```
# policies/pii-redact.yaml
name: pii-redact
when:
  capability_prefix: ["docs.", "reasoning.", "audio."]
action: redact
detectors: [email, phone, ssn]
```

Loaded at boot, re-loadable with SIGHUP. Policies can block, redact, or require approval. v0.1 ships one example policy.

### 6.4 Models ( `models/*.{onnx,tflite,bin}` )

Downloaded by `scripts/download-default-models.sh`. SHA-256 verified by ant-models on first use. Three tiers (minimum / standard / full) match different RAM budgets. See `MODELS.md`.

### 6.5 Golden set ( `golden/*.yaml` )

Ground-truth evaluation data for `eval.golden_run`. Pass / fail numbers from these drive the regression protection on new builds.

---

## 7. End-to-end: one HTTP call traced

What happens when a browser calls `POST /api/v1/call/vision/detect`:




Stages:

1. **Route match** (microseconds): axum's radix router finds the handler.
2. **Capability lookup** (microseconds): one HashMap read.
3. **Policy evaluation** (microseconds): short-circuits on first match.

4. **Supervisor dispatch** (milliseconds): hands the payload to the worker handle.
5. **IPC round trip** (varies by worker; see per-worker table in `MODELS.md`).
6. **Audit write** (microseconds; SQLite WAL).
7. **Event broadcast** (microseconds).
8. **Response**: back out through axum.

Total overhead on top of the worker's own work: typically under 2 ms.

---

## 8. Pipeline execution

A pipeline is an ordered list of stages in `arikizu.yaml`:

```
pipeline:
  - stage: ingest
    capability: audio.transcribe
    input: { file: "{{ input.file }}" }

  - stage: redact
    capability: guard.redact
    input: { text: "{{ previous.transcript }}" }

  - stage: human-review
    require_approval: true
    reviewer_role: operator

  - stage: summarize
    capability: reasoning.summarize
    input: { text: "{{ redact.text }}" }
```

The pipeline executor:

1. Walks stages in order.
2. Resolves `{{ ... }}` references against previous stage outputs.
3. On `require_approval: true`, enqueues an approval and blocks the stage until the dashboard says approve or reject.
4. On any stage failure, writes a `pipeline.failed` audit row and stops (no partial writes).
5. On success, writes a `pipeline.completed` audit row with the full per-stage trace.

A pipeline is a first-class citizen of the audit log; you can replay any past run from the recorded trace.

---

## 9. Boot sequence

When `arikizu run` starts (either from the shell or from systemd):

```
t=0.00  parse CLI flags, load arikizu.yaml
t=0.01  ant-common::config::load_or_default()
t=0.02  ant-router::CapabilityRegistry::load("capabilities.toml")
t=0.03  ant-feedback::AuditLog::open("data/audit.sqlite") -> migrations
t=0.04  ant-feedback::FeedbackManager::new(audit)
t=0.05  ant-devices::DeviceInventory::detect()
t=0.06  ant-models::ModelRegistry::load_manifest() -> SHA verify
t=0.10  Supervisor::spawn_all()  -> N Python subprocesses, N Unix sockets
         - test:        50 ms to answer
         - vision:    2-5 s (loads the vision model to memory)
         - audio:     1-3 s (loads VAD)
         - docs:      <1 s
         - timeseries: <1 s
         - memory:    <100 ms
         - guard:     <100 ms
         - eval:      <100 ms
         - reasoning: 0 ms (no model load until first call)
t=5-10  all workers ACK'd ready
t=5.10  ant-http::serve(0.0.0.0:7000)
         - log "serving demo sandbox at /demo"
         - log "serving operator dashboard at /"
         - log "HTTP server listening"
t=5.11  ready to serve requests
```

On a Pi 4 the cold start is ~15 s. On a laptop ~5 s.

---

## 10. Deployment architecture

Three distribution tiers, same binary, different packaging.




Full tier description: `DISTRIBUTION.md`.

---

## 11. Scaling model

Arikizu is deliberately single-device. Why not multi-node:

- The target customer is a factory, nurse's station, ship engine room, temple conservator. They do not have a Kubernetes cluster.
- Edge latency guarantees come from running the model next to the sensor, not from distributing it.
- Multi-node would need auth, service discovery, mTLS, and a non-single-user mental model, all of which contradict "install with one command and forget."

Horizontal within one device: the supervisor can spawn multiple instances of the same worker (`vision-1`, `vision-2`, ...) and the router load-balances calls across them. Vertical: pick a bigger device (Pi 4 to Pi 5 to Jetson Orin Nano to Jetson AGX Orin), same binary, no code change.

For the rare multi-device case (factory with 20 lines, hospital with 50 beds), the pattern is one Arikizu per device, each shipping events to a central log. That is a separate product, not a feature here.

---

## 12. What is NOT in the architecture

Explicit non-features. Each saves a lot of complexity.

| Not here | Why |
|---|---|
| Multi-tenant auth | One device, one owner. Add OIDC later only if needed. |
| Multi-node scheduling | Kubernetes isn't the deployment unit for a Pi. |
| Model training | Edge is for inference; training stays off-device. |
| Cloud sync | Opt-in, not default. Audit log can be forwarded to Loki/Grafana. |
| GUI model-marketplace browser | `scripts/download-default-models.sh` + CLI is enough today. |
| Plugin system | A new worker IS a plugin. Drop a package in `workers/`, done. |

## 13. File-by-file map

Where every important file lives:

```
arikizu/
  Cargo.toml                  workspace manifest
  arikizu.yaml.example        reference pipeline + worker config
  capabilities.toml           29 capabilities declared here
  Makefile                    one-liner wrappers around cargo+uv

  crates/<9 crates>/          Rust workspace (see section 3)

  workers/<10 workers>/       Python workspace (see section 5)
  workers/arikizu-worker-sdk/ the SDK every worker imports

  ui/static/index.html        operator dashboard served at /
  ui/static/assets/           logo, favicon

  demo/web/*.html             developer sandbox served at /demo/
  demo/fixtures/              sample inputs for the demos
  demo/raspberrypi/           Pi-only probe + camera scripts

  policies/*.yaml             declarative rules
  golden/*.yaml               eval ground truth

  models/*.onnx               downloaded on demand
  data/audit.sqlite           created on first run

  scripts/                    bootstrap, check_prereqs, model download
  packaging/                  debian/, pi-gen/, systemd/

  docs/                       this folder
```

## 14. Where to go next

- `DEMO.md` : walk through every demo view with expected output
- `MODELS.md` : per-worker model catalog and backend selection
- `HOW_IT_RUNS.md` : stack diagram and systemd integration
- `DISTRIBUTION.md` : the three install tiers
- `SPEC.md` : non-fit list, five-point qualification, verticals
- `SECURITY.md` : threat model, dependency validation